task1

September 27, 2026

[10]:
```
import numpy as np
import pandas as pd
from collections import Counter

def calculate_entropy(y):
    _, counts = np.unique(y, return_counts=True)

    probabilities = counts / len(y)
    probabilities = probabilities[probabilities > 0]
    return -np.sum(probabilities * np.log2(probabilities))

def calculate_information_gain(y, y_left, y_right):
    weight_left = len(y_left) / len(y)
    weight_right = len(y_right) / len(y)

    entropy_parent = calculate_entropy(y)
    entropy_children = (weight_left * calculate_entropy(y_left)) +␣
 ↪(weight_right * calculate_entropy(y_right))

    return entropy_parent - entropy_children
```

[ ]:
```
from platform import node

from numpy import unique_values
import pandas as pd
import numpy as np
from collections import Counter

class Node:
    def __init__(self, feature = None, threshold=None, left=None, right=None,␣
 ↪value=None, is_categorical=False):
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.value = value
        self.is_categorical = is_categorical
```

```
    def is_leaf_node(self):
        if self.value is not None:
            return True
        else:
            return False

    def __str__(self):
        if self.is_leaf_node():
            return f"Leaf(value={self.value})"

        branch_type = "==" if self.is_categorical else "<="
        left_tree = str(self.left).replace("\n", "\n    ")
        right_tree = str(self.right).replace("\n", "\n    ")

        return (
            f"Node(feature={self.feature}, "
            f"condition={branch_type} {self.threshold})\n"
            f"  Left:\n    {left_tree}\n"
            f"  Right:\n    {right_tree}"
        )

class MultiClassDecisionTree:
    def __init__(self, min_samples_split=2, max_depth=100):
        self.min_samples_split = min_samples_split
        self.max_depth = max_depth
        self.root = None

    def fit(self, X, y):
        if not isinstance(X, pd.DataFrame): X = pd.DataFrame(X)
        if not isinstance(y, pd.Series): y = pd.Series(y)

        self.root = self._build_tree(X, y, depth=0)

    def _build_tree(self, X, y, depth):
        n_samples, n_features = X.shape
        n_labels = len(np.unique(y))

        if(depth >= self.max_depth or n_samples < self.min_samples_split or␣
 ↪n_labels==1):
            leaf_value = self.most_common_label(y)
            return Node(value=leaf_value)

        best_feat, best_thresh, is_cat = self._get_best_split(X, y)

        if best_feat is None:
            leaf_value = self.most_common_label(y)
```

```
            return Node(value=leaf_value)

        if is_cat:
            left_idxs = X[best_feat] == best_thresh
        else:
            left_idxs = X[best_feat] <= best_thresh

        right_idxs = ~left_idxs

        left_child = self._build_tree(X[left_idxs], y[left_idxs], depth + 1)
        right_child = self._build_tree(X[right_idxs], y[right_idxs], depth + 1)

        return Node(best_feat, best_thresh, left_child, right_child,␣
↪is_categorical=is_cat)

    def _get_best_split(self, X, y):
        best_gain = -1
        split_idx, split_threshold, is_categorical = None, None, False

        for feature in X.columns:
            column_data = X[feature]
            if not pd.api.types.is_numeric_dtype(column_data):
                unique_values = column_data.unique()
                for value in unique_values:
                    left_idxs = column_data == value
                    right_idxs = ~left_idxs

                    if(len(y[left_idxs]) == 0 or len(y[right_idxs]) == 0):
                        continue

                    gain = calculate_information_gain(y, y[left_idxs],␣
↪y[right_idxs])
                    if gain > best_gain:
                        best_gain = gain
                        split_idx = feature
                        split_threshold = value
                        is_categorical = True

            else:
                unique_values = np.sort(column_data.unique())
                midpoints = (unique_values[:-1] + unique_values[1:]) / 2

                for threshold in midpoints:
                    left_idxs = column_data <= threshold
                    right_idxs = column_data > threshold

                    if len(y[left_idxs]) == 0 or len(y[right_idxs]) == 0:
```

```
                    continue

                gain = calculate_information_gain(y, y[left_idxs], y[right_idxs])

                if gain > best_gain:
                    best_gain = gain
                    split_idx = feature
                    split_threshold = threshold
                    is_categorical = False

        return split_idx, split_threshold, is_categorical

    def most_common_label(self, y):
        counter = Counter(y)
        return counter.most_common(1)[0][0]

    def predict(self, X):
        if not isinstance(X, pd.DataFrame): X = pd.DataFrame(X)
        return np.array([self._traverse_tree(x, self.root) for _, x in X.iterrows()])

    def _traverse_tree(self, x, node):
        if node.is_leaf_node():
            return node.value

        feature_val = x[node.feature]

        if node.is_categorical:
            if feature_val == node.threshold:
                return self._traverse_tree(x, node.left)
            return self._traverse_tree(x, node.right)
        else:
            if feature_val <= node.threshold:
                return self._traverse_tree(x, node.left)
            return self._traverse_tree(x, node.right)
```

[12]:
```
df = pd.read_csv("drug200.csv")
X = df[["Age", "Sex", "BP", "Cholesterol", "Na_to_K"]]
y = df["Drug"]
```

[13]:
```
import numpy as np

split_index = int(len(X) * 0.67)

X_train, X_test = X.iloc[:split_index], X.iloc[split_index:]
y_train, y_test = y.iloc[:split_index], y.iloc[split_index:]
```

```python

tree = MultiClassDecisionTree(min_samples_split=2, max_depth=5)
tree.fit(X_train, y_train)

predictions = tree.predict(X_test)

def calculate_accuracy(y_true, y_pred):
    y_true_array = np.array(y_true)
    correct_predictions = np.sum(y_true_array == y_pred)
    return correct_predictions / len(y_true_array)

accuracy = calculate_accuracy(y_test, predictions)
print(f"Model Accuracy: {accuracy * 100:.2f}%")
```

```
Node(feature=Age, condition=<= 51.5)
  Left:
    Leaf(value=drugA)
  Right:
    Leaf(value=drugB)
Node(feature=Cholesterol, condition=== HIGH)
  Left:
    Leaf(value=drugC)
  Right:
    Leaf(value=drugX)
Node(feature=BP, condition=== LOW)
  Left:
    Node(feature=Cholesterol, condition=== HIGH)
      Left:
        Leaf(value=drugC)
      Right:
        Leaf(value=drugX)
  Right:
    Leaf(value=drugX)
Node(feature=BP, condition=== HIGH)
  Left:
    Node(feature=Age, condition=<= 51.5)
      Left:
        Leaf(value=drugA)
      Right:
        Leaf(value=drugB)
  Right:
    Node(feature=BP, condition=== LOW)
      Left:
        Node(feature=Cholesterol, condition=== HIGH)
          Left:
            Leaf(value=drugC)
          Right:
            Leaf(value=drugX)
```

```
      Right:
        Leaf(value=drugX)
Node(feature=Na_to_K, condition=<= 14.627)
  Left:
    Node(feature=BP, condition=== HIGH)
      Left:
        Node(feature=Age, condition=<= 51.5)
          Left:
            Leaf(value=drugA)
          Right:
            Leaf(value=drugB)
      Right:
        Node(feature=BP, condition=== LOW)
          Left:
            Node(feature=Cholesterol, condition=== HIGH)
              Left:
                Leaf(value=drugC)
              Right:
                Leaf(value=drugX)
          Right:
            Leaf(value=drugX)
  Right:
    Leaf(value=drugY)
Model Accuracy: 96.97%
```

[14]:
```
tree = MultiClassDecisionTree(max_depth=5, min_samples_split=2)
tree.fit(X, y)

new_patient = pd.DataFrame({
    'Age': [35],
    'Sex': ['M'],
    'BP': ['HIGH'],
    'Cholesterol': ['NORMAL'],
    'Na_to_K': [16.5]
})

prediction = tree.predict(new_patient)
print(f"Recommended Drug: {prediction[0]}")
```

```
Node(feature=Age, condition=<= 50.5)
  Left:
    Leaf(value=drugA)
  Right:
    Leaf(value=drugB)
Node(feature=Cholesterol, condition=== HIGH)
  Left:
    Leaf(value=drugC)
  Right:
```

```
    Leaf(value=drugX)
Node(feature=BP, condition=== LOW)
  Left:
    Node(feature=Cholesterol, condition=== HIGH)
      Left:
        Leaf(value=drugC)
      Right:
        Leaf(value=drugX)
  Right:
    Leaf(value=drugX)
Node(feature=BP, condition=== HIGH)
  Left:
    Node(feature=Age, condition=<= 50.5)
      Left:
        Leaf(value=drugA)
      Right:
        Leaf(value=drugB)
  Right:
    Node(feature=BP, condition=== LOW)
      Left:
        Node(feature=Cholesterol, condition=== HIGH)
          Left:
            Leaf(value=drugC)
          Right:
            Leaf(value=drugX)
      Right:
        Leaf(value=drugX)
Node(feature=Na_to_K, condition=<= 14.8285)
  Left:
    Node(feature=BP, condition=== HIGH)
      Left:
        Node(feature=Age, condition=<= 50.5)
          Left:
            Leaf(value=drugA)
          Right:
            Leaf(value=drugB)
      Right:
        Node(feature=BP, condition=== LOW)
          Left:
            Node(feature=Cholesterol, condition=== HIGH)
              Left:
                Leaf(value=drugC)
              Right:
                Leaf(value=drugX)
          Right:
            Leaf(value=drugX)
  Right:
    Leaf(value=drugY)
```

```
Recommended Drug: drugY
```